

ProgressRadar：本地优先的个人进度雷达与复盘系统

3099404236

Independent Developer · 2026-08-02

github.com/3099404236/progress-radar

摘要 ProgressRadar 是一个本地优先的个人项目进度系统。它用桌面快速粘贴收集零散文本，经 AI 结构化维度、阶段、周期、时间线与成就，同时允许非线性推进和人工确认。v2.1 进一步加入锦囊任务板、灵犀阁、个人主页增量扫描，以及面向私人 GitHub 仓库的完整备份链路。备份链路将普通代码与数据复制制到独立快照，将本机密钥转换为 Windows DPAPI CurrentUser 密文，附带 Git 历史恢复包，并通过计划任务、状态网页和双色悬浮球持续报告成功、失败、运行中或过期状态。本文档说明系统边界、交互模型、备份设计和实机验证结果。

关键词 local-first · progress tracking · AI organization · private backup · desktop application

问题与设计目标

个人项目很少沿直线前进：一次实验可能让计划回到前一阶段，一段阅读笔记可能同时影响多个项目，投稿或考试又会开启新的周期。传统待办清单擅长回答“做没做”，却不容易保留“为什么变化、处于哪个阶段、跨过了什么里程碑”。

ProgressRadar 把日常输入视为事件，而不是强迫用户先选择表格字段。用户粘贴一段文本后，AI 返回 update、create、evolve、confirm 或 skip 等结构化动作；最终落库前仍由本机规则和必要的人工确认约束。

设计原则：收集入口要短，数据解释要可追溯，阶段允许跳跃，私人数据默认留在本机；公开代码与完整私人快照必须是两条独立发布路径。

v2.1 的工作面

工作面	解决的问题	主要数据
进度雷达	把文本沉淀为维度、阶段、周期和时间线	progress.json
锦囊任务板	保留临时计划；完成后自动转成一次进展提交	tasks.json
灵犀阁	按“卷”收藏灵感、心得和可复用观点	viewpoints.json
主页扫描	比较网页快照，识别项目或论文的实质新增	snapshot + history
成就系统	把里程碑、连续活跃和洞察变成可见反馈	achievements.json

表 1 ProgressRadar v2.1 的五个互补工作面。

公开仓库提供代码、示例配置、PDF 与通用备份工具；任务、观点、扫描历史、浏览器缓存和明文密钥不进入公开提交。

系统结构与数据流

快速粘贴与仪表盘	→	Python JS Bridge	→	AI 动作与本地规则
任务 / 灵感 / 主页	→	原子 JSON 数据层	→	阶段 / 周期 / 成就

表 2 主交互与数据落库链路。前端和 Python 在同一桌面进程中通信，不对外暴露业务 HTTP 端口。

事件处理

一次普通提交先读取现有维度与周期，再构造 AI 上下文。返回结果通过动作分派器处理：update 更新已有维度，create 建立新维度，evolve 提议新的阶段结构，confirm 应用映射，skip 则保留原始档案而不制造虚假进展。跨维度关系、时间线事件、周期事件与成就字段相互独立，避免一个字段失败时丢弃整次输入。

本地数据层

文件类别	写入策略	公开策略
进度与成就	临时文件写完后原子替换；保留 schema 迁移	可选公开样例
任务与观点	线程锁保护；按 ID 增删改	默认不公开
主页快照与历史	快照原子替换，历史追加 JSONL	默认不公开
API 密钥	环境变量或 secrets.local.py	仅 DPAPI 密文进入私有备份
WebView 缓存	运行时自动生成	公开和私有远端都排除

表 3 不同数据的持久化与发布边界。

为什么保留原始记录

AI 分类会受提示词、模型版本和上下文影响。系统同时保留原始输入与结构化结果，使后续可以重新归类、解释阶段变化，或在模型暂时不可用时继续收集。这里的“本地优先”不是完全离线：当启用 DeepSeek 时，提交内容会发送给所配置的模型服务；用户应据此决定哪些文本适合提交。

交互模型：从待办到复盘

锦囊任务板

锦囊面向“现在先记下来”的内容。卡片支持待办、完成、筛选与删除。完成卡片时，系统会构造明确的进展文本再次送入分类流程；提示词要求这类主动提交不能被轻易 skip，因此一次完成动作既结束待办，也成为可复盘的项目事件。

灵犀阁

灵犀阁把非任务型的学习心得单独保存。用户可建立多个“卷”，为长观点命名、复制或删除。它避免将每个灵感强行解释成进度，同时保留未来与项目连接的可能。

个人主页扫描

主页扫描器抓取 HTML、移除脚本与样式文本，并与上次快照比较。内容哈希相同则记录 no_change；出现新增行时，AI 结合完整页面上下文生成一句更新描述，再把该描述送入进度系统。首次扫描与增量扫描分开处理，失败时保留历史记录但不伪造成功快照。

页面比较是文本级启发式方法。动态渲染、导航调整或同义改写可能造成误报；因此扫描结果被当作“候选进展”，而不是不可撤销的事实。

成就与阶段

里程碑成就来自可计算规则，洞察成就可以由 AI 建议，自定义成就由用户创建。阶段环形图使用每条记录的 phase_index 汇总，而不是假设只能从左到右推进；周期视图则区分“当前一轮”和历史全部记录。

记录想法	→	建立锦囊	→	完成行动	→	沉淀进展
页面更新	→	增量检测	→	生成描述	→	阶段 / 成就

表 4 两条典型路径最终汇入同一个可复盘进度模型。

私有完整备份与可观测性

备份不是直接在公开仓库执行 `git add all`，而是在本机维护一个独立快照仓库。每轮运行先递归复制允许的文件，清理远端快照中已不存在的旧文件，再分别处理密钥和 Git 历史。

项目目录	→	过滤与快照	→	私人 GitHub 仓库
secrets.local.py	→	DPAPI CurrentUser	→	protected 密文
源仓库 .git	→	git bundle -all	→	recovery 历史包
计划任务	→	status.json	→	状态网页 / 双色球

表 5 完整备份链路及其三个输出：工作树、受保护配置和可恢复历史。

防误操作与恢复

- 快照根目录不得与源目录相同，也不得互相嵌套；驱动器根目录会被拒绝。
- 缓存、虚拟环境、临时测试区和“本机私用”ZIP 被显式排除。
- 接近 GitHub 单文件 100 MB 硬限制的文件会在推送前失败并报告路径。
- 互斥锁禁止两个备份并发运行；电脑休眠错过任务后由 `StartWhenAvailable` 补跑。
- 状态 JSON 先写临时文件再原子替换，悬浮球不会读到半份结果。
- `source-history.bundle` 可在新目录中克隆或验证，用来恢复公开仓库已有提交历史。

DPAPI CurrentUser 密文只能在同一 Windows 用户上下文中直接解密。因此它防止私有仓库中的密钥以明文暴露，但不能替代系统账户安全，也不能作为跨机器恢复密钥的唯一方案。

状态语义

颜色	状态	含义
绿色	success	最近一次运行成功，含 <code>pushed</code> 或 <code>no_changes</code>
蓝色	running	当前正在复制、提交或推送
红色	failed	最近一次运行失败，状态页保留错误摘要
黄色	stale	成功记录超过计划间隔两倍加一小时
灰色	unknown	尚无记录、配置缺失或状态文件不可读

表 6 悬浮球右边的备份状态；左半边继续显示机器人链路。

无变化判定

快照清单不写入每次运行都会变化的生成时间；已存在的 DPAPI 密文会先在内存中解密并与当前明文比较，内容相同则复用原密文。这样，文件、Git 历史和受保护配置都没有实质变化时，第二轮运行不会制造空提交，而是推送确认远端后返回 `no_changes`。若源文件被删除，快照中的对应旧文件也会安全清理。

监控与恢复路径

备份脚本把 `running`、`success` 或 `failed` 原子写入 `status.json`，并保留最近成功时间、提交号、规模、下次计划和错误摘要。本机状态服务同时读取手机机器人监控与备份状态；悬浮球点击打开综合页面，右键可请求计划任务立即运行。恢复时可直接克隆私人快照取得工作树；若要重建源仓库已有历史，则验证并克隆 `recovery/source-history.bundle`，再把快照中的未提交文件覆盖回工作树。

状态页只监听 127.0.0.1，不把备份详情公开到局域网或互联网。私人远端使用 Git Credential Manager 已有登录状态，配置文件不保存 GitHub token。

验证、局限与复现

2026-08-02 实机验证

验证项	结果	判据
隔离双轮演练	通过	首轮 pushed, 次轮 no_changes, 提交号不变
DPAPI 往返	通过	密文解开后与源文件逐字节一致
明文密钥排除	通过	快照树无 project/secrets.local.py
Git 历史包	通过	git bundle verify 成功
真实首次快照	通过	141 个文件, 89,825,778 字节, 1 个受保护文件
远端一致性	通过	本地快照 HEAD 与私人远端 main 完全一致
计划任务	通过	PT2H; 任务可用时补跑; 重复触发由互斥锁跳过
综合监控	通过	状态页同时读取机器人链路 with 备份状态

表 7 本版本交付前的自动与实机检查。数值描述的是一次快照规模, 不包含被排除的运行缓存。

本地复现

1. 安装依赖并配置 API key; 2. 运行 main.py 验证桌面交互; 3. 运行 private_backup/Test-PrivateBackup.ps1 做隔离备份自检; 4. 准备私人仓库后运行安装器; 5. 打开本机 8788 状态页; 6. 用 scripts/build-local.ps1 重新生成本文 PDF。

局限

当前 JSON 数据层适合单用户桌面场景, 不适合多设备同时写入。AI 归类与主页摘要仍依赖外部模型质量。文本级网页差分无法完整理解客户端渲染页面。GitHub 私有仓库降低了公开泄露风险, 但不等于离线、不可删除或零信任备份; 关键资料应保留第二份离线副本。DPAPI 密文也需要额外的跨设备恢复策略。

结论

ProgressRadar v2.1 把“收集—解释—行动—复盘—备份”连成一个本地桌面闭环。公开发布链路保持可审计与可复现, 私人链路则保存完整工作状态并持续报告健康度。两者分离, 使项目既能公开迭代, 又不必牺牲个人数据边界。